Smart Health — Détection de Chutes et Tracking de Posture (Patient Monitoring)

Réaliser par : CLERVIL Adonaï et ATOUAILAA Asmaa, encadé par : Mohamed EL BRAK
MSITBD, S2, Département Génie Informatique
FST de Tanger

1. Introduction

1.1. L'avènement de la santé intelligente

Le secteur de la santé connaît aujourd'hui une transformation progressive grâce aux technologies numériques. L'intelligence artificielle, l'Internet des Objets et la vision par ordinateur offrent des possibilités concrètes pour améliorer la prise en charge des patients, notamment dans les pays où l'automatisation du secteur médical reste encore limitée. Dans ce contexte, la notion de Smart Health prend tout son sens : il ne s'agit pas simplement d'innover pour innover, mais de mettre la technologie au service de situations réelles qui coûtent des vies.

1.2. Le défi sociétal du vieillissement et des chutes

Au Maroc comme dans de nombreux pays en développement, le vieillissement de la population s'accélère, mais les infrastructures de surveillance médicale ne suivent pas toujours ce rythme. Les personnes âgées vivant seules ou dans des conditions de surveillance insuffisante sont particulièrement exposées aux risques de chutes. Et pourtant, une chute n'est pas toujours mortelle en elle-même, c'est souvent l'absence de prise en charge rapide qui transforme un incident évitable en une situation dramatique. Une fracture non détectée, des heures passées au sol sans aide, un traumatisme ignoré, voilà les conséquences réelles d'un système de surveillance défaillant.

C'est ce constat simple mais frappant qui est à l'origine de notre projet.

1.3. Les limites des solutions existantes

Les solutions qui existent déjà pour détecter les chutes reposent le plus souvent sur des objets connectés portés par la personne, bracelets, pendentifs, montres intelligentes. Sur le papier, l'idée est bonne. Dans la pratique, c'est une autre histoire.

D'abord, beaucoup de personnes âgées oublient tout simplement de porter ces dispositifs, surtout la nuit, précisément quand le risque de chute est le plus élevé. Ensuite, porter un objet en permanence devient vite inconfortable, et certains finissent par l'abandonner après quelques semaines. Mais le problème le plus profond est peut-être psychologique : pour beaucoup de personnes âgées, accepter de porter un pendentif d'alarme, c'est accepter publiquement qu'elles ne sont plus autonomes. Et ça, beaucoup refusent de le faire.

Il fallait donc trouver une autre approche, une qui protège sans contraindre, et qui fonctionne même quand la personne n'y pense pas.

1.4. L'IoT et l'IA comme paradigme de rupture

La réponse que nous avons explorée repose sur une idée assez directe : utiliser une caméra et de l'intelligence artificielle pour surveiller la personne sans qu'elle ait besoin de faire quoi que ce soit. Pas de bracelet, pas de pendentif, pas de geste à mémoriser. La caméra observe, l'algorithme analyse, et si quelque chose d'anormal se produit, une alerte est envoyée automatiquement.

1.5. Présentation et objectifs du projet

Notre projet SmartHealth est né d'un constat simple : le secteur de la santé au Maroc manque encore cruellement d'automatisation, et des vies pourraient être sauvées si des outils technologiques accessibles étaient mis en place. Nous avons voulu poser une première brique concrète, un système capable de détecter automatiquement une chute, d'enregistrer la scène pour permettre une vérification humaine, et d'alerter immédiatement un soignant ou un proche.

Le système que nous avons développé poursuit quatre objectifs concrets :

Premièrement, détecter toute chute automatiquement en analysant en temps réel la position et le mouvement du corps humain à travers une caméra.

Deuxièmement, enregistrer une vidéo de l'incident pour que le soignant puisse vérifier lui-même ce qui s'est passé avant d'intervenir.

Troisièmement, alerter immédiatement les personnes responsables par email, par une alerte sonore et depuis le tableau de bord, sans aucune action manuelle requise.

Et quatrièmement, faire tout cela sans que la personne surveillée ait besoin de porter quoi que ce soit.

La suite de cet article présente l'état de l'art des systèmes existants, les choix technologiques que nous avons faits et pourquoi, puis l'architecture complète du système et les résultats obtenus.

2. État de l'art

2.1. Systèmes existants de détection de chutes

Avant de concevoir notre système, nous avons commencé par explorer ce qui existait déjà. Et des solutions, il en existe : smartwatches, bracelets connectés, pendentifs d'alarme, systèmes à accéléromètres. À première vue, le problème semblait presque résolu. Mais en regardant de plus près, on réalise rapidement que ces systèmes partagent tous une même lacune fondamentale : ils détectent, mais ils ne montrent pas.

C'est là que notre approche se distingue. Les systèmes portables sont efficaces sur le papier, limités en pratique. La majorité des travaux existants dans la littérature reposent sur des capteurs portés sur le corps. Hussain et al. [3] proposent par exemple un algorithme basé sur l'apprentissage automatique, combinant des classifieurs SVM et KNN sur des données d'accéléromètres, qui atteint une précision de 99,98% sur le dataset SisFall. Un chiffre impressionnant. Mais cette précision ne résout pas une question fondamentale : comment savoir si l'alerte déclenchée correspond à une vraie chute ou à un faux positif ?

Dans le même esprit, Naushin et al. [4] développent un bracelet intelligent intégrant accéléromètre, gyroscope et microcontrôleur, capable de classifier en temps réel les chutes par rapport aux activités quotidiennes. Les auteurs parlent eux-mêmes de solution "non invasive", mais le dispositif reste un objet physique que la personne doit porter en permanence, avec tous les problèmes que cela implique : oubli, inconfort, rejet psychologique.

Drouot [1] va encore plus loin dans la spécialisation, en développant pour des patients atteints de myosite à inclusions une orthèse instrumentée équipée d'encodeurs articulaires et d'IMU. Les résultats sont remarquables, 100% de détection, mais le dispositif est complexe, coûteux, et totalement inadapté à un déploiement grand public.

2.2. Les systèmes par vision : une piste prometteuse

Face à ces limitations, certains chercheurs se sont tournés vers la vision par ordinateur. Nguyen et al. [2] proposent une architecture basse consommation pour un système de détection de chutes implémenté

sur une plateforme embarquée Zynq-7000. Leurs travaux montrent qu'il est possible de traiter un flux vidéo en temps réel sans GPU dédié.

C'est dans cette direction que nous avons choisi de travailler.

2.3. Limitations des systèmes existants

Qu'ils soient basés sur des capteurs portables ou sur la vision par ordinateur, les systèmes existants partagent une même limite : ils envoient une alerte, mais ils ne donnent aucun moyen de vérifier si cette alerte est justifiée.

Or dans la réalité, un soignant qui reçoit une alerte a besoin de savoir ce qui s'est passé avant d'intervenir. Est-ce une vraie chute ? La personne est-elle consciente ? A-t-elle besoin d'une ambulance ou simplement d'aide pour se relever ? Sans élément visuel, impossible de répondre à ces questions.

C'est précisément ce manque que notre système SmartHealth adresse. En plus de détecter la chute, il enregistre automatiquement une vidéo de l'incident et la rend accessible depuis le tableau de bord Node-RED permettant au soignant de voir de ses propres yeux ce qui s'est passé avant d'intervenir. Cette fonctionnalité de levée de doute visuelle, absente des solutions existantes, constitue selon nous l'apport le plus concret de notre travail.

Critère	Capteurs portables	Vision par ordinateur	Notre système
Intrusivité	Élevée	Nulle	Nulle
Précision	Très élevée	Élevée	Élevée
Faux positifs	Possibles	Possibles	Vérifiables
Levée de doute vidéo	✗	✗	✓
Dépendance au port	Oui	Non	Non
Alerte temps réel	Oui	Partielle	Oui
Déploiement	Individuel	Environnemental	Environnemental

Figure 1- Comparatif entre les différentes approches

2.4. Évolution des technologies d'estimation de pose

Pour construire notre système, nous avions besoin d'un outil capable d'extraire en temps réel les points clés du corps humain à partir d'une simple caméra. Plusieurs bibliothèques existent pour cela : OpenPose, YOLO-Pose, MediaPipe.

2.4.1. OpenPose : puissant mais trop lourd

OpenPose, développé par Carnegie Mellon University, est probablement la solution la plus connue et la plus citée dans la littérature pour l'estimation de pose humaine. Il utilise des réseaux de neurones convolutifs profonds et des *Part Affinity Fields* pour détecter simultanément plusieurs personnes dans une même image, avec une précision remarquable.

Mais voilà le problème : OpenPose est extrêmement gourmand en ressources. Sans GPU dédié, il devient inutilisable en temps réel. Pour un projet comme le nôtre, destiné à fonctionner sur une machine standard sans matériel spécialisé, c'était rédhibitoire. On ne peut pas demander à un système de surveillance déployé dans une maison de dépendre d'une carte graphique haut de gamme.

2.4.2. YOLO-Pose : rapide mais complexe

Les versions récentes de YOLO, notamment YOLOv8-pose, ont étendu les capacités de détection d'objets à l'estimation de pose. L'architecture en une seule passe (*single-stage*) lui confère une vitesse impressionnante, et des versions allégées commencent à rendre le déploiement embarqué envisageable.

Cela dit, YOLO reste une solution pensée avant tout pour des environnements avec GPU, et son intégration dans un pipeline Python orienté santé demande un niveau de configuration qui dépasse ce que notre projet nécessitait. De plus, la documentation et les exemples disponibles sont moins accessibles pour un usage médical simple.

2.4.3. MediaPipe : le choix qui s'est imposé

C'est en lisant les travaux de Said Salimo [7], un projet de détection de chute basé sur l'analyse des points du squelette en 2D, que nous avons découvert MediaPipe Pose de Google. Et c'est cette bibliothèque que nous avons choisie, pour plusieurs raisons concrètes.

D'abord, MediaPipe fonctionne efficacement sur CPU seul. Bazarevsky et al. [5] montrent dans l'article original de BlazePose, le moteur derrière MediaPipe Pose, que le système détecte 33 points clés du corps humain et tourne à plus de 30 images par seconde sur un simple smartphone, sans GPU. Pour nous, c'était exactement ce qu'il fallait.

Ensuite, MediaPipe est simple à intégrer en Python. Quelques lignes de code suffisent pour avoir un squelette humain complet détecté en temps réel.

Enfin, et c'est peut-être le point le plus important pour notre contexte, MediaPipe est gratuit et open source. Dans un projet qui vise à être accessible et déployable sans coût prohibitif, c'est un critère qui compte.

2.4.4. Pourquoi le squelette 2D et pas 3D ?

Comme Said Salimo [7] le souligne dans ses travaux, travailler en 2D plutôt qu'en 3D présente un avantage majeur dans notre contexte : la simplicité et la légèreté. La détection 3D offre certes une précision supérieure, mais elle implique des capteurs de profondeur (comme le Kinect) et une complexité de traitement bien plus grande. Pour un système destiné à fonctionner avec une caméra standard dans un environnement domestique, le 2D est un compromis raisonnable et suffisant.

Parmi les 33 points détectés par MediaPipe, nous n'en utilisons que 5, ceux qui sont réellement pertinents pour la détection de chute : la tête, le centre des épaules, le centre du bassin, la cheville gauche et la cheville droite. C'est une simplification volontaire qui réduit la charge de calcul sans dégrader la qualité de la détection.

Critère	OpenPose	YOLO-Pose	MediaPipe
Précision	Très élevée	Élevée	Bonne
Vitesse sur CPU	Lente	Moyenne	Rapide
Légèreté	Lourde	Moyenne	Très légère
Intégration Python	Complexe	Moyenne	Simple
Déploiement sans GPU	✗	Partiel	✓
Points détectés	25	17	33
Adapté à notre projet	✗	⚠ Partiel	✓

Figure 2- Comparatif entre OpenPose, YOLO-Pose, MediaPipe Pose

2.4.5. Choix de MediaPipe Pose

Au final, MediaPipe Pose s'est imposé comme le choix le plus adapté à nos contraintes : machine standard, développement Python, déploiement sans infrastructure complexe. C'est sur cette base que nous avons construit notre pipeline de détection, en nous inspirant de l'approche de Said Salimo [7] que nous avons étendue avec une infrastructure IoT complète incluant MQTT, Node-RED, Firebase et un système d'alerte automatique par email.

2.5. Les protocoles de communication IoT

Une fois la chute détectée, il faut transmettre l'alerte. Et là, une question simple se pose : comment envoyer ce message le plus vite possible, de manière fiable, sans surcharger le système ?

Dans un cours d'IoT, la réponse qu'on entend le plus souvent c'est : MQTT. Mais avant de l'adopter automatiquement parce que c'est "le standard du cours", nous avons voulu comprendre pourquoi et la comparaison avec HTTP nous a très vite convaincus que ce choix était le bon.

2.5.1. HTTP : familier mais inadapté

HTTP est le protocole qu'on connaît tous, c'est celui qui fait fonctionner les sites web. Son fonctionnement est simple : un client envoie une requête, un serveur répond. C'est ce qu'on appelle le modèle Request/Response.

Le problème, c'est que ce modèle n'est pas fait pour des événements critiques en temps réel. Chaque requête HTTP transporte un en-tête lourd, plusieurs centaines d'octets de métadonnées, même quand le message lui-même ne contient que quelques mots. Et surtout, HTTP ne permet pas nativement au serveur d'envoyer une information au client sans que celui-ci ne l'ait demandée. Dans notre cas, ça voudrait dire que le tableau de bord Node-RED devrait constamment interroger le script Python pour savoir si une chute a été détectée, ce qu'on appelle du *polling*, ce qui est à la fois inefficace et énergivore.

Les chiffres parlent d'eux-mêmes. Craggs [6] montre que pour 100 messages envoyés, HTTP génère 554 600 octets de trafic réseau, contre seulement 44 748 octets pour MQTT, soit 12 fois moins. Et le temps de réponse moyen d'HTTP reste bloqué à 289 ms quel que soit le nombre de messages, là où MQTT descend à 43 ms pour une connexion active. Dans un contexte médical où chaque seconde compte, cette différence n'est pas négligeable.

2.5.2. MQTT : conçu pour exactement ce dont nous avions besoin

MQTT repose sur un modèle complètement différent : le Publish/Subscribe. Le principe est simple : un *publisher* publie un message sur un *topic*, et tous les *subscribers* abonnés à ce topic le reçoivent automatiquement, sans avoir rien demandé.

Dans notre système, ça se traduit concrètement comme ça : dès qu'une chute est détectée, le script Python publie un message sur le topic smarthealth/falls. Node-RED, abonné à ce topic, le reçoit instantanément et déclenche toutes les alertes en parallèle : dashboard, email, audio, Firebase. Zéro polling, zéro latence inutile, zéro message manqué.

Ce qui nous a aussi convaincu, c'est le système de QoS, Quality of Service, que MQTT propose nativement en trois niveaux :

- QoS 0 : le message est envoyé une fois, sans confirmation
- QoS 1 : le message est envoyé au moins une fois, avec accusé de réception
- QoS 2 : le message est délivré exactement une fois, sans perte ni doublon

Nous avons choisi le QoS 2 pour les alertes de chute. Parce que dans notre cas, une alerte perdue peut vouloir dire une personne laissée sans aide. Et une alerte dupliquée peut provoquer une panique inutile. QoS 2 élimine ces deux risques.

2.5.3. Mosquitto comme broker local

Pour le broker, l'intermédiaire qui gère la distribution des messages, nous avons utilisé Eclipse Mosquitto, déployé localement sur la même machine. C'est une solution légère, open source, et parfaitement adaptée à un déploiement local sans dépendance à un service cloud externe. La communication se fait en local sur 127.0.0.1:1883, ce qui garantit une latence minimale entre la détection et la transmission de l'alerte.

Critère	HTTP	MQTT
Modèle	Request/Response	Publish/Subscribe
En-tête minimum	~200 octets	2 octets
Trafic 100 messages	554 600 octets	44 748 octets
Temps de réponse moyen	289 ms	43 ms
Push natif	✗	✓
QoS garanti	✗	✓ (3 niveaux)
Adapté à notre projet	✗	✓

Figure 3- Comparatif entre HTTP et MQTT

3. Description de l'architecture globale

3.1. Justification des choix technologiques

Concevoir un système de détection de chutes, c'est résoudre plusieurs problèmes en même temps : détecter le mouvement, analyser le comportement, transmettre l'information, et afficher une alerte, le tout en temps réel. Chaque brique technologique que nous avons choisie répond à une contrainte précise, et aucun de ces choix n'a été fait au hasard.

Nous avons travaillé en binôme sur l'ensemble du projet, ce qui nous a permis d'avoir une vision complète de chaque composant et de comprendre comment ils s'articulent entre eux.

Python + MediaPipe constituent le cœur du système. Comme nous l'avons vu dans la section précédente, MediaPipe Pose [5] permet d'extraire 33 points clés du squelette humain en temps réel sur CPU seul. Python, avec ses bibliothèques NumPy et OpenCV, nous a permis d'implémenter l'ensemble du pipeline de détection de manière claire et maintenable. Nous nous sommes appuyés sur les travaux de Said Salimo [7] comme base, que nous avons ensuite étendue avec toute la partie IoT.

Le filtre de Kalman a été retenu pour une raison simple : détecter une chute ne se résume pas à regarder où se trouve la tête. Il faut aussi savoir à quelle vitesse elle descend. Le filtre de Kalman nous permet d'estimer, pour chaque point du squelette, non seulement sa position mais aussi sa vitesse et son accélération, trois grandeurs qui, combinées, permettent de distinguer une vraie chute d'une personne qui se baisse pour ramasser quelque chose.

MQTT avec Mosquitto s'est imposé comme protocole de communication pour les raisons que nous avons détaillées en section 2.3 — légèreté, faible latence, modèle Publish/Subscribe adapté aux événements critiques, et QoS 2 pour garantir la livraison exacte de chaque alerte.

Node-RED a été choisi pour orchestrer les alertes. La partie qui nous a demandé le plus de travail dans Node-RED était justement l'intégration du lecteur vidéo dans le dashboard, faire en sorte que la vidéo de l'incident soit automatiquement chargée et lisible depuis le tableau de bord, avec le bon format, le bon chemin, et le bon codec. Ce n'était pas trivial, mais c'est précisément cette fonctionnalité qui donne au système sa valeur ajoutée réelle : permettre au soignant de voir ce qui s'est passé avant d'intervenir.

Firebase complète l'architecture en assurant la persistance des événements dans le Cloud. Chaque incident est enregistré avec son horodatage, consultable depuis n'importe quel appareil connecté.

3.2. Schéma fonctionnel du système

Le système SmartHealth s'articule autour de quatre couches fonctionnelles :

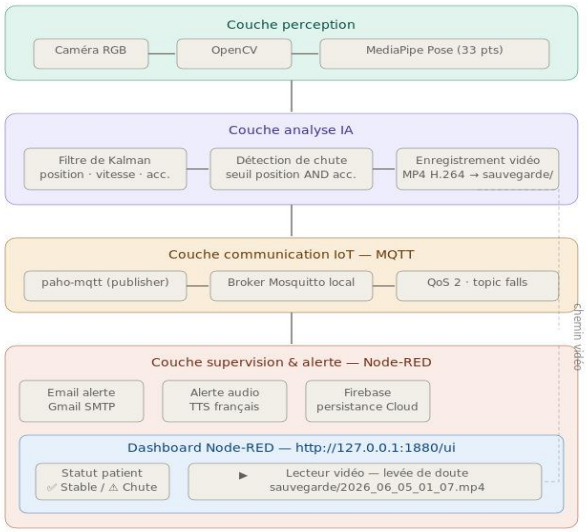


Figure 4- Architecture globale du projet

3.3. Modèle logique d'intégration

Le système fonctionne de manière entièrement automatique, sans aucune intervention humaine entre la détection et l'alerte. Voici comment les composants collaborent concrètement :

Le script Python tourne en continu, analysant chaque frame de la caméra. À chaque image,

MediaPipe extrait les coordonnées des 5 points clés du corps. Le filtre de Kalman prédit l'état de chaque point : position, vitesse, accélération. Deux algorithmes s'exécutent : l'un analyse la position verticale de la tête par rapport aux chevilles, l'autre analyse l'accélération verticale des articulations. Une chute n'est confirmée que lorsque les deux alarmes sont actives simultanément. Ce double critère réduit significativement les faux positifs.

Dès la confirmation, le script crée un fichier vidéo MP4 encodé en H.264/avc1 dans le dossier sauvegarde/, horodaté à la seconde. L'enregistrement continue après la fin de l'événement pour capturer la situation complète.

Une seule alerte MQTT est ensuite publiée par incident avec ce message JSON :

```
 {
    "status": "fall_detected",
    "timestamp": time.strftime("%Y-%m-%d
%H:%M:%S"),
    "video_file": nom_fichier_video,
    "seuil_p": seuil_position,
    "seuil_a": seuil_acceleration
 }
```

Node-RED reçoit le message et orchestre cinq actions en parallèle : mise à jour du dashboard en rouge, chargement automatique de la vidéo dans le lecteur, envoi d'un email d'alerte, lecture d'une alerte vocale en français, et enregistrement de l'incident dans Firebase.

Quand la situation se stabilise, le script publie :

```
{
   "status": "Stable",
   "timestamp": time.strftime("%Y-%m-%d
%H:%M:%S")
}
```

Le dashboard repasse en vert, mais la vidéo du dernier incident reste visible à l'écran. C'est un choix délibéré : même après le retour à la normale, le soignant doit pouvoir consulter ce qui s'est passé.

Ce moment où on a vu pour la première fois le système fonctionner de bout en bout, c'était une vraie satisfaction. Pas seulement parce que ça marchait techniquement, mais parce qu'on réalisait qu'on avait transformé une idée en quelque chose de concret, capable de répondre à un vrai problème que des gens vivent au quotidien.

4. Partie IoT : Capteurs, Communication MQTT, Node-RED et Cloud

4.1. Architecture IoT du système

Notre système SmartHealth suit une architecture IoT classique à trois niveaux : perception, transport, application. Mais ce qui la rend intéressante, c'est que chaque niveau a été choisi et configuré pour répondre à des contraintes réelles, pas juste pour cocher des cases dans un cahier des charges.

La caméra observe. MQTT transporte. Node-RED agit. C'est aussi simple que ça sur le papier, mais la mise en œuvre, elle, a demandé du travail.

4.2. Capteurs et actionneurs

4.2.1. Le capteur : une simple caméra

Dans notre système, le capteur c'est la caméra et c'est précisément là que réside l'un de nos choix les plus importants. Pas de bracelet, pas de capteur à porter, pas de matériel spécialisé. Une caméra RGB standard suffit.

MediaPipe extrait les coordonnées des 5 points clés du squelette à chaque frame, sans jamais transmettre les images brutes sur le réseau, ce qui préserve la confidentialité des données de la personne surveillée.

4.2.2. Les actionneurs

Les actionneurs sont tous les composants qui réagissent quand une chute est détectée :

Actionneur	Technologie	Rôle
Dashboard web	Node-RED ui_template	Affichage visuel alerte
Lecteur vidéo	HTML5 + Node-RED httpStatic	Levée de doute visuelle
Synthèse vocale	Node-RED ui_audio (TTS)	Alerte sonore français
Email d'alerte	Gmail SMTP port 465	Notification soignants
Base de données Cloud	Firebase Realtime DB	Persistance événements
Enregistreur vidéo	OpenCV VideoWriter H.264	Sauvegarde locale

Figure 5- Listes des actionneurs et leur rôle

4.3. Communication MQTT

4.3.1. Configuration de Mosquitto

La configuration du broker Mosquitto a pris un peu de temps au départ. Ce n'est pas l'outil le plus complexe, mais il fallait s'assurer que le broker était bien démarré avant le script Python, que les ports étaient ouverts, et que Node-RED pointait vers la bonne adresse. Une fois ces détails réglés, la communication est devenue parfaitement stable.

Paramètres finaux :

Adresse broker : 127.0.0.1
Port : 1883
Client ID : nodered_sh
Protocole : MQTT v4 (3.1.1)
Session : Clean session
Keep-alive : 60 secondes

4.3.2. Topic et structure des messages

Nous utilisons un seul topic : smarthealth/falls

Deux types de messages transitent sur ce topic selon l'état du système.

En cas de chute détectée :

```
{
    "status": "fall_detected",
    "timestamp": time.strftime("%Y-%m-%d %H:%M:%S"),
    "video_file": nom_fichier_video,
    "seuil_p": seuil_position,
    "seuil_a": seuil_acceleration
}
```

Au retour à la normale :

```
{
    "status": "Stable",
    "timestamp": time.strftime("%Y-%m-%d %H:%M:%S")
}
```

4.3.3. QoS et mécanisme anti-doublon

Nous utilisons le QoS 2 pour les alertes de chute, livraison garantie exactement une fois. C'est le niveau le plus strict de MQTT, justifié par le caractère médical de l'alerte.

En complément, un flag applicatif alerte_envoyee dans le script Python garantit qu'une seule alerte est publiée par incident, même si la chute est détectée sur plusieurs frames consécutifs.

Ce double mécanisme, QoS au niveau protocole et flag au niveau applicatif, assure une robustesse maximale dans la transmission des alertes.

4.4. Node-RED : de la réception à l'action

4.4.1. Le flux

Le flux Node-RED reçoit chaque message MQTT sur le topic smarthealth/falls via le nœud Fall Alert, le parse en un objet JSON, puis le route vers six branches parallèles :

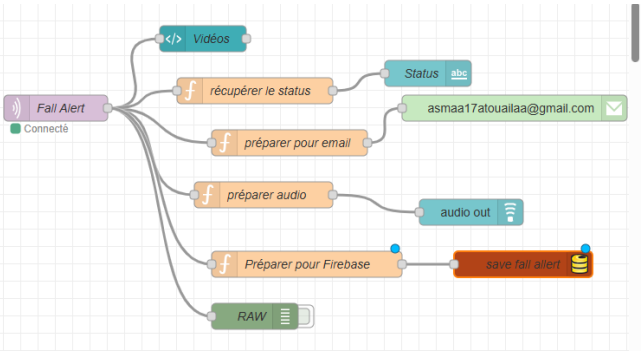


Figure 6- Flux Node-RED

1. Vidéos : affiche en temps réel le statut de l'alerte et la dernière vidéo enregistrée dans le dashboard.

2. Récupérer le status → Status : extrait le statut du message (fall_detected ou Stable)

et l'affiche dans le dashboard.

3. Préparer pour email → asmaa17atouailaa@gmail.com : en cas de chute, prépare et envoie un email d'alerte contenant le nom du fichier vidéo.

4. Préparer audio → audio out : en cas de chute, génère un message vocal d'alerte diffusé via le dashboard.

5. Préparer pour Firebase → save fall allert : en cas de chute uniquement, formate les données (status, video_file, timestamp) et les sauvegarde dans Firebase Realtime Database sous le chemin falls/<timestamp>.

6. RAW : nœud de débogage permettant de visualiser le message brut reçu.

La logique de routage est simple : en cas de chute, les 6 sorties sont actives. En cas de retour à la normale, seules les sorties 1 et 2 sont actives, pour mettre à jour le statut et conserver la vidéo du dernier incident à l'écran.

4.4.2. La fonctionnalité vidéo, notre contribution principale

La partie qui nous a demandé le plus de travail dans Node-RED, c'est précisément celle qui différencie notre système de tout ce qui existait déjà : l'intégration du lecteur vidéo dans le dashboard.

Il a fallu résoudre plusieurs problèmes techniques en série.

1. D'abord, le format vidéo : les navigateurs ne lisent pas les fichiers AVI, nous avons donc dû passer à MP4 avec le codec H.264.
2. Ensuite, l'accessibilité du fichier : Node-RED ne peut pas servir des fichiers locaux par défaut, nous avons configuré la propriété httpStatic dans settings.js pour pointer vers le dossier du projet.
3. Enfin, le chemin relatif : le script Python calcule dynamiquement le chemin de la vidéo et le transmet dans le message MQTT, pour que le dashboard sache exactement quel fichier charger.

Le résultat final : dès qu'une chute est détectée, la vidéo de l'incident apparaît automatiquement dans le dashboard, prête à être lue par le soignant pour vérifier si l'alerte est justifiée.

4.4.3. Le dashboard

Le tableau de bord accessible à http://127.0.0.1:1880/ui affiche en temps réel :

- Le statut du patient — "Stable" en vert ou " CHUTE DETECTEE" en rouge
- Le bandeau d'alerte dynamique change de couleur selon l'état
- Le lecteur vidéo, chargé automatiquement avec la vidéo du dernier incident
- Les métadonnées : nom du fichier et horodatage précis de l'incident

4.5. Cloud : Firebase

L'intégration Firebase est réalisée via un nœud Firebase dans Node-RED.

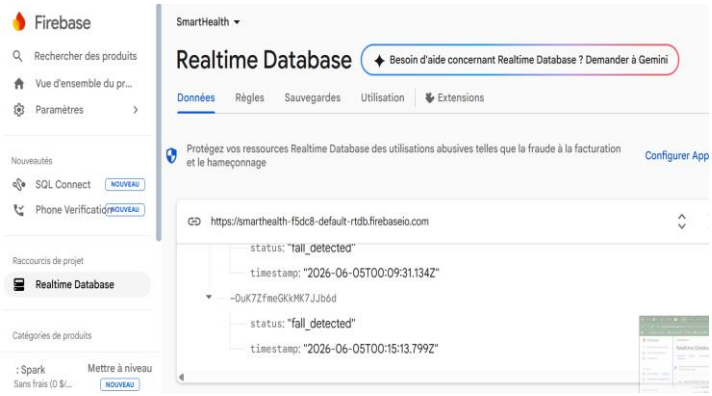


Figure 7- Dashboard de Firebase

Chaque incident est ainsi enregistré dans le Cloud avec son horodatage, consultable depuis n'importe quel appareil connecté, ce qui ouvre la possibilité d'un suivi longitudinal des chutes d'un patient sur la durée.

4.6. Synthèse

Voici un récapitulatif des différents composants du projet et leur rôle.

Composant	Rôle	Technologie
Caméra	Capteur	OpenCV
Script Python	Détection IA	MediaPipe + Kalman
Mosquitto	Broker MQTT	Eclipse Mosquitto
Node-RED	Orchestration	Node-RED v3
Dashboard	Interface soignant	Node-RED Dashboard
Gmail SMTP	Notification email	node-red-node-email
TTS Audio	Alerte sonore	ui_audio
Firebase	Historique Cloud	HTTP REST API

Figure 8- Tableau de synthèse des composants

5. Partie IA

5.1. Objectifs de l'IA

La brique logicielle dédiée à l'Intelligence Artificielle constitue le noyau décisionnel de notre système. Son rôle premier est de garantir la mise en œuvre d'une approche de surveillance strictement non intrusive. Contrairement aux dispositifs matériels classiques portés par les patients (*wearables*), qui génèrent un taux d'abandon élevé en raison de la stigmatisation ou de l'oubli de recharge, l'implémentation de la vision par ordinateur permet de s'affranchir de toute contrainte physique directe sur le sujet. L'objectif technique consiste à analyser, de manière asynchrone et en temps réel, un flux vidéo standard provenant d'une caméra ambiante afin de modéliser et de suivre la dynamique corporelle de l'individu dans son espace de vie.

Au-delà d'une simple tâche de détection de présence ou de soustraction de fond, le système opère une analyse géométrique structurelle approfondie de la silhouette humaine. À partir des images capturées, l'algorithme extrait des caractéristiques biomécaniques fondamentales pour en déduire des indicateurs physiques pertinents.

Ces indicateurs ciblent principalement deux variables cinétiques déterminantes :

- La posture spatiale : l'orientation géométrique et l'inclinaison des segments corporels par rapport aux axes de référence de la scène.
- La vitesse vectorielle : la vélocité des mouvements verticaux et horizontaux calculée entre les trames successives de la vidéo.

Ces données extraites à la périphérie (*Edge computing*) sont ensuite immédiatement traitées par un modèle logique à double seuillage. Ce dernier combine l'analyse de position et le calcul d'accélération pour classifier l'activité du patient, permettant ainsi de discriminer avec précision une chute accidentelle d'une action de la vie quotidienne (comme s'asseoir ou s'allonger de manière contrôlée).

Dès qu'une anomalie est validée, cette brique IA a pour objectif final d'enregistrer localement la séquence visuelle de l'impact, de la finaliser de manière synchrone sous un format vidéo universel et hautement compatible, puis d'encapsuler ces métriques ainsi que le lien d'accès au fichier dans un message structuré JSON. Cet envoi déclenche instantanément la chaîne d'alerte et de levée de doute sur l'infrastructure IoT.

5.2. Données utilisées

La mise en œuvre de notre système de vision ne requérant aucun capteur physique sur le patient, le flux de données exploité est exclusivement d'origine vidéographique. Le projet étant réalisé en mode simulé, les données d'entrée sont constituées de flux vidéo standards au format RGB, soit issus d'une webcam locale émulant une caméra de surveillance, soit à partir de vidéos pré-enregistrées de simulations de chutes.

Le traitement de ces données brutes s'articule autour de deux types de représentations :

1. Les données d'entrée brutes (Pixels) : Le flux vidéo capture des images matricielles successives. Ces images servent d'intrants directs au framework de Deep Learning MediaPipe Pose. Ce modèle de réseau de neurones convolutifs (CNN) analyse la topologie de l'image pour prédire la pose humaine.
2. Les points d'intérêt biomécaniques (Landmarks) : Bien que MediaPipe génère

un ensemble de 33 points clés tridimensionnels (X, Y, Z) couvrant l'intégralité du corps, notre approche logicielle applique un filtre sélectif pour optimiser les ressources computationnelles. S'inspirant des travaux de la littérature scientifique (notamment Mundher & Zhong), seules les coordonnées spatiales 2D (X, Y) de 5 points d'intérêt critiques sont conservées et historisées d'un frame à l'autre :

- La tête (représentant l'extrémité supérieure et le vecteur principal d'accélération lors d'un impact),
- Le centre de la ligne des épaules,
- Le centre du bassin (assimilsé au centre de gravité de la silhouette),
- La cheville droite,
- La cheville gauche.

Ce passage d'un flux de pixels lourd à un ensemble restreint de 5 vecteurs de coordonnées numériques constitue la base de données dynamique qui va alimenter nos algorithmes de classification et de filtrage temporel.

5.3. Modèle choisi et logique algorithmique

Pour transformer les données géométriques brutes fournies par l'IA en une décision d'alerte fiable, notre système déploie une architecture algorithmique séquentielle en trois étapes : l'interfaçage avec le flux, le filtrage cinétique des coordonnées, et la classification par arbre de décision sémantique.

A. Acquisition dynamique et intégration de la webcam

Contrairement au prototype initial de la littérature qui se limitait à l'analyse passive de fichiers vidéo pré-enregistrés, nous avons implémenté un module d'acquisition flexible basé sur la bibliothèque OpenCV (cv2.VideoCapture). Le système peut ainsi commuter de manière transparente entre deux sources :

- Le mode analyse/simulation : lecture de fichiers .mp4 ou .avi issus de datasets publics de chutes pour valider l'algorithme.
- Le mode déploiement réel : capture instantanée du flux d'une webcam locale, émulant le comportement d'une caméra de surveillance IP. Les trames y sont capturées image par image à une fréquence standard, permettant une réactivité maximale.

B. Suivi cinétique et lissage par Filtre de Kalman

Dès que MediaPipe Pose extrait les coordonnées spatiales 2D des 5 points d'intérêt (tête, épaules, bassin, chevilles), celles-ci sont transmises à un ensemble de filtres de Kalman discrets et récursifs. L'introduction de cet algorithme mathématique répond à deux objectifs critiques :

- Le lissage et la robustesse aux bruits : Les prédictions des réseaux de neurones peuvent souffrir de micro-oscillations ou de pertes temporaires de tracking (dues à des variations d'éclairage de la webcam). Le filtre de Kalman estime l'état réel de la position en minimisant l'erreur quadratique, assurant la continuité du suivi même en cas d'occultation brève.
- L'extraction de l'accélération : À chaque instant t, le filtre de Kalman modélise le vecteur d'état de chaque point comprenant sa position (x), sa vélocité (v) et son accélération (a). C'est cette accélération verticale (a_y), calculée de manière purement algorithmique, qui sert de descripteur dynamique pour la détection d'impact.

C. Algorithme de classification par double seuillage logique

La décision finale de l'état du patient ne repose pas sur un modèle de Machine Learning de classification boîte noire (souvent trop lourd pour une exécution en périphérie), mais sur une méthode déterministe combinant deux critères physiques stricts :

- Le seuil de position (S_p) : L'algorithme évalue si la coordonnée verticale des points clés franchit une ligne critique de basculement. Dans notre repère où l'axe Y est orienté vers le bas, une chute implique que :

  $Y_{bassin} > S_p$.

- Le seuil d'accélération (S_a) : Pour discriminer une chute d'une action normale (comme s'asseoir lentement sur le sol), l'accélération verticale estimée par le filtre de Kalman doit dépasser une valeur limite d'impact :

$$a_y > S_a$$

La règle d'activation de l'alarme :

Pour éliminer les faux positifs induits par des activités dynamiques de la vie quotidienne ou des positions statiques basses, le système valide l'état de détresse par l'intersection logique des deux alarmes :

$$Alarme\ final = Alarme\ Posture \cap Alarme\ Accélération$$

Dès que cette condition logique est vérifiée, le système bascule instantanément en mode de sauvegarde active. Un fichier vidéo conteneur .mp4 est immédiatement initialisé en périphérie à l'aide du codec H.264 (avc1), hautement compatible avec les technologies Web. L'algorithme continue d'enregistrer les trames d'impact ainsi qu'un tampon temporel post-chute ajustable (fin_mouvement) servant à capter le contexte immédiat de l'incident (absence de mouvement, tentative de redressement).

Une fois le compteur épuisé, le flux d'écriture est formellement interrompu et le fichier vidéo est finalisé et clos sur le stockage local de la passerelle *Edge*. C'est uniquement à la suite de cette clôture physique garantissant l'intégrité du fichier que la brique décisionnelle IA encapsule de manière synchrone le nom unique du fichier vidéo, l'horodatage précis et les métriques physiques dans un payload JSON. Ce message est alors publié sur le réseau IoT via le protocole MQTT à destination du serveur Node-RED pour l'affichage temps réel et la levée de doute sur le tableau de bord des secours.

5.4. Résultats obtenus et validations expérimentales

L'évaluation de notre prototype a été réalisée en environnement simulé afin de valider l'intégralité de la chaîne de traitement, depuis la capture vidéo en périphérie (*Edge*) jusqu'à la réactivité de l'interface de contrôle (*Dashboard*).

A. Performance de l'extraction des points clés et du filtrage

L'intégration du framework MediaPipe Pose couplé à notre réseau de filtres de Kalman discrets démontre une excellente robustesse face aux contraintes de l'environnement (changements de luminosité, occultations partielles de la silhouette). Le lissage récursif élimine efficacement le bruit haute fréquence (le "tremblement" des points) lors des phases stationnaires et restitue une courbe d'accélération d'impact propre et exploitable pour le double seuillage.



Figure 9- Détection de chute

B. Comportement du double seuillage logique et synchronisation

L'expérimentation confirme la pertinence du modèle déterministe. Lors d'actions quotidiennes de type "s'asseoir rapidement" ou "se baisser", bien que la vitesse vectorielle augmente brièvement, le centre de gravité (bassin) ne franchit pas le seuil de position minimal (S_p). Inversement, lors d'une chute réelle, l'intersection logique des deux critères déclenche instantanément la phase d'enregistrement de sécurité.

La modification architecturale visant à retarder l'émission de la notification MQTT s'est révélée cruciale : le système attend la stabilisation

complète du patient au sol et l'expiration du compteur de contexte pour exécuter la fermeture physique du descripteur vidéo (video_writer.release()). Aucun phénomène de corruption de fichier ou de trames tronquées n'a été observé.

C. Intégration IoT et Levée de doute sur le Dashboard Node-RED

Dès la clôture du fichier, la publication du payload JSON sur le broker MQTT est instantanée. Côté serveur, l'isolation de la route de stockage via la directive httpStatic: httpStatic: 'C:/Users/hp/Desktop/Master/cours_securite_IT/S2/IoT/FallDetection/DAIOProject/sauvegarde', supprime tout conflit de routage HTTP.

L'implémentation de la persistance de l'état graphique au sein du scope AngularJS du nœud Template ($root.dernierevideoId) résout le problème de l'actualisation. Lors de la réception de l'alerte, l'interface utilisateur rafraîchit dynamiquement le lecteur HTML5 natif sans nécessiter de rechargement manuel de la page. De plus, lors du retour automatique du script Python à l'état de veille "Stable", la séquence vidéo de l'impact reste ancrée à l'écran, garantissant au personnel soignant la possibilité d'effectuer une levée de doute complète.

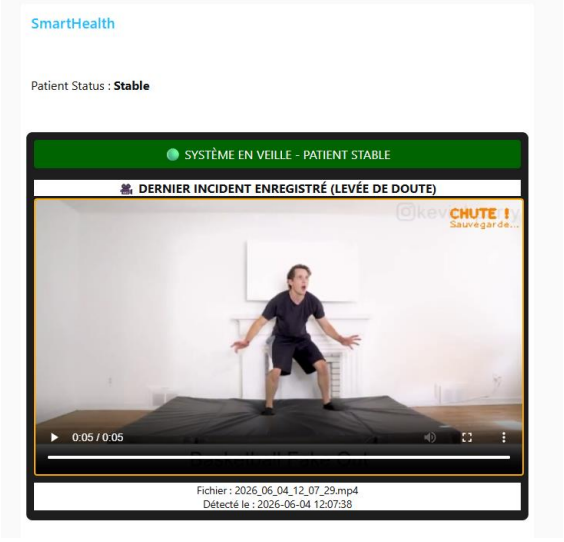


Figure 10-Dashboard Node-Red

6. Résultats et démonstration

L'évaluation expérimentale de notre prototype a permis de valider l'intégralité de la chaîne de valeur du système, depuis la détection de l'impact en périphérie (*Edge*) jusqu'à la mise à disposition de la séquence vidéo pour le personnel soignant sur le tableau de bord (*Dashboard*).

6.1. Validation du traitement local et interface d'analyse unifiée

La première phase de validation concerne la brique d'Intelligence Artificielle s'exécutant localement en Python. Par souci d'optimisation des ressources computationnelles de la machine hôte, le système n'instancie qu'une seule et unique fenêtre graphique principale. Celle-ci centralise l'intégralité du retour visuel et du suivi algorithmique.

Le framework MediaPipe Pose extrait et connecte en temps réel le squelette de la silhouette à partir du flux vidéo brut. En tâche de fond, le filtre de Kalman estime de manière continue la position du bassin et l'accélération verticale. Lors d'une simulation d'impact, le franchissement simultané des deux seuils physiques bascule instantanément le système en état d'alerte. Ce changement d'état est immédiatement notifié visuellement par l'incrustation en rouge du message critique "CHUTE DETECTEE !" tout en haut de cette unique fenêtre d'analyse.



Figure 11-Fenêtre d'affichage unifiée affichant le squelette MediaPipe et l'incrustation de l'alerte lors d'une simulation de chute.

6.2. Analyse et résolution des anomalies de communication HTTP

La mise au point de la passerelle entre le script Python et l'interface Node-RED a nécessité la résolution de deux comportements anomaux majeurs liés à l'asynchronisme initial du système :

- L'erreur HTTP 416 (Range Not Satisfiable) : Lors des premiers tests (notamment avec le format .webm), l'émission de la notification MQTT se déclenchait dès le début de la chute. Le navigateur tentait alors d'interroger immédiatement le fichier alors que Python continuait d'y écrire les trames. Le lecteur HTML5, ne trouvant pas les en-têtes de fin de fichier nécessaires à l'indexation, renvoyait une erreur 416.
- L'erreur HTTP 404 (Not Found) : Des conflits de routage sur l'URL d'interrogation (/ui/...mp4) survenaient lorsque le chemin d'accès du lecteur vidéo s'emmêlait avec les routes internes du Dashboard au lieu de pointer vers le répertoire statique dédié.

La restructuration de la logique algorithmique a permis d'éradiquer ces erreurs. En forçant le script Python à enregistrer l'intégralité de la chute ainsi que la temporisation de contexte (cpt_fin_mouvement), puis à exécuter impérativement la clôture physique du fichier (video_writer.release()) avant d'émettre le message MQTT, l'intégrité de la vidéo est garantie à 100 % au moment où le réseau s'en empare.

6.3. Démonstration du Dashboard et validation du streaming partiel

Une fois le payload JSON reçu par Node-RED, le Dashboard AngularJS met à jour dynamiquement ses composants. Grâce au déplacement de la directive ng-src directement dans la balise principale <video>, le lecteur HTML5 vide son cache et charge instantanément la nouvelle séquence vidéo dès qu'une chute survient, sans nécessiter aucun rafraîchissement manuel de la page. De plus, l'utilisation du scope global $root fige la dernière vidéo à l'écran à des fins de levée de doute, même lorsque le système Python repasse en mode veille et transmet un statut "Stable".

L'analyse des flux réseaux dans les outils de développement du navigateur confirme le succès de cette architecture. L'interrogation des fichiers MP4 successifs (encodés via le codec compatible avc1) se stabilise et valide l'établissement d'un protocole de streaming partiel.

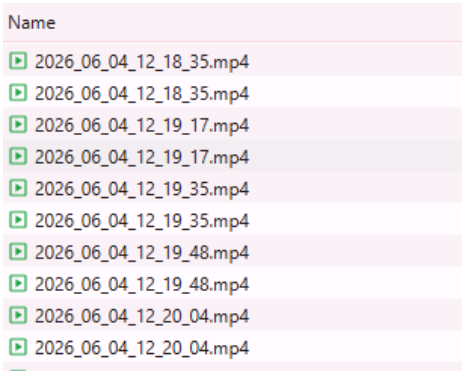


Figure 12- Liste des requêtes HTTP successives des vidéos de chutes captées par le navigateur.

Le serveur Node-RED, configuré avec la route statique isolée, répond aux requêtes du navigateur par un code de succès 206 Partial Content et transmet l'en-tête Accept-Ranges: bytes.

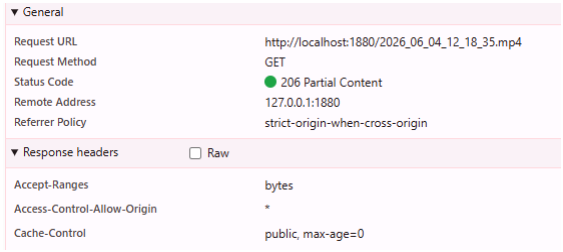


Figure 13- Détail des en-têtes HTTP de la réponse, confirmant le protocole d'échange par morceaux (Code 206).

Cette négociation réseau réussie prouve que le fichier est lu par blocs d'octets au fur et à mesure de sa lecture par l'utilisateur. Cela permet au personnel soignant non seulement de visionner l'incident en direct, mais aussi de naviguer librement dans la barre de progression temporelle de la vidéo pour analyser précisément les circonstances de la chute.

7. Discussion, limites et perspectives

7.1. Discussion

La validation expérimentale de notre prototype met en évidence l'efficacité d'une architecture décentralisée (*Edge Computing*) pour la surveillance non intrusive. L'optimisation du script Python à travers une unique interface graphique d'analyse prouve qu'il est possible de concilier la légèreté computationnelle d'un traitement local et la pertinence d'un modèle déterministe.

De plus, la résolution des anomalies de communication HTTP représente un apport technique majeur de ce travail. En indexant et en clôturant proprement le flux vidéo en amont de la publication MQTT, le système garantit l'intégrité de la séquence transmise au Dashboard. Enfin, la persistance graphique via le scope global $root sous AngularJS offre une interface de levée de doute résiliente pour le personnel soignant, qui conserve la preuve visuelle de l'impact même lorsque le système repasse automatiquement à l'état de veille "Stable".

7.2. Limites de la méthode

Bien que l'implémentation de notre méthode de détection de chute à partir des points du squelette en 2D ait donné des résultats satisfaisants en démonstration, il est nécessaire d'en analyser les limites intrinsèques, qui constituent autant d'axes d'amélioration pour les développements futurs.

7.2.1. Limitations dans la détection des points (Couche IA)

L'étape initiale d'extraction des caractéristiques dépend entièrement des performances du framework MediaPipe Pose. Or, la détection des points du squelette peut être fortement influencée par des conditions d'éclairage défavorables ou par le bruit visuel dans l'environnement du patient, ce qui est susceptible de fausser les prédictions du réseau de neurones.

Par ailleurs, les phénomènes de camouflage et d'obstruction physique posent une limite majeure : si la personne est en partie masquée par un meuble ou si l'axe optique de la caméra est obstrué, la détection perd en précision, omet des points clés critiques ou génère des coordonnées topologiques complètement fausses.

7.2.2. Limitations dans le suivi des points (Couche Filtrage)

Le suivi cinétique assuré par le filtre de Kalman fait face à des difficultés mathématiques dans les situations complexes où les mouvements sont rapides ou chaotiques. Le filtre de Kalman standard repose sur des hypothèses strictes de linéarité. Or, une chute humaine est un phénomène dynamique hautement non linéaire ; par conséquent, lors de l'impact, le filtre classique n'est plus en mesure d'estimer l'état cinétique du système de manière optimale.

De surcroît, les problèmes d'occultation mentionnés précédemment affectent directement la robustesse du suivi. Bien que notre implémentation du filtre de Kalman soit capable de maintenir l'estimation de la position d'un point lors d'une occultation brusque et temporaire, cette capacité s'amenuise avec le temps. Plus l'occultation de la silhouette est longue, moins le filtre reçoit de mesures réelles pour se corriger, entraînant une dérive mathématique et une falsification croissante de l'estimation.

7.2.3. Limitations dans la reconnaissance de la chute (Couche Logique)

Les faiblesses accumulées lors de la détection et du suivi se répercutent inévitablement sur la brique décisionnelle, augmentant le taux de faux positifs. Au-delà du bruit de mesure, la logique algorithmique souffre de deux biais structurels :

- L'effet de perspective et d'éloignement : La taille apparente du corps à l'écran diminue à mesure que le patient s'éloigne de la caméra. Plus la silhouette est petite, plus elle devient susceptible de franchir de manière erronée le seuil de la fonction PersonFallingDown_position.
  Actuellement, nos seuils de position et

d'accélération sont initialisés de façon rigide en fonction de la taille de la matrice de l'image, et non en fonction de la taille anthropométrique réelle de la personne.

- Les limites des seuils déterministes : Notre modèle à double seuillage peine à discriminer certaines actions quotidiennes complexes. À l'instar de la fonction PersonFallingDown_position qui engendre un faux positif si le patient s'allonge volontairement sur le sol, la fonction PersonFallingDown_acceleration valide une alerte erronée si l'individu effectue un mouvement vertical trop brusque, comme se baisser rapidement pour ramasser un objet.

7.3. Perspectives et évolutions futures

Pour pallier ces limitations, plusieurs perspectives d'évolution s'ouvrent afin d'industrialiser le dispositif :

- Évolution mathématique du filtrage : Remplacer le filtre de Kalman linéaire par des approches adaptées aux dynamiques non linéaires, telles que le Filtre de Kalman Étendu (EKF) ou le Filtre de Particules (PF), afin de modéliser fidèlement la phase de rupture cinétique de l'impact.
- Seuils dynamiques et adaptatifs : Modifier les fonctions PersonFallingDown pour que l'initialisation des seuils physiques s'ajuste dynamiquement en temps réel selon le calcul de la taille de la silhouette de la personne à l'écran, corrigeant ainsi le biais de l'éloignement.
- Fusion de capteurs et vision 3D : Intégrer des caméras de profondeur (type infrarouge ou *Time-of-Flight*) pour obtenir des coordonnées réelles dans l'espace en trois dimensions (X, Y, Z). Cela permettrait de s'affranchir des contraintes d'éclairage (fonctionnement nocturne) et de calculer des distances absolues indépendantes de la perspective de l'image.
- Enrichissement de l'interface IoT : Connecter un module audio bidirectionnel (via le protocole WebRTC) au sein du Dashboard Node-RED, permettant d'ajouter une levée de doute sonore (écoute des gémissements ou appel vocal) en complément du flux vidéo MP4 H.264.

8. Conclusion

Le travail présenté dans ce document a permis de concevoir, d'implémenter et de valider un prototype complet de système de détection de chutes à domicile pour personnes vulnérables. L'objectif principal, qui consistait à proposer une solution de surveillance strictement non intrusive et transparente pour l'utilisateur, a été pleinement atteint grâce à l'exploitation de techniques modernes de vision par ordinateur et à l'architecture IoT distribuée.

Au cœur de la brique décisionnelle locale, le couplage entre l'extraction de squelette bidimensionnel par MediaPipe Pose et le traitement cinétique par filtre de Kalman a démontré qu'un modèle déterministe à double seuillage est tout à fait viable en périphérie (*Edge Computing*). Cette approche permet de s'affranchir de infrastructures de calcul lourdes et de préserver la confidentialité des données, l'analyse s'effectuant en tâche de fond au sein d'une interface d'affichage unique et optimisée.

La transition entre l'algorithme Python local et l'infrastructure réseau a mis en lumière des défis d'ingénierie logicielle critiques. La résolution des anomalies de flux a prouvé l'importance d'une synchronisation stricte entre le cycle de vie du fichier vidéo sur le disque et l'émission des alertes MQTT. En garantissant la clôture physique du fichier conteneur `.mp4` (H.264 / `avc1`) avant la notification, le système assure un streaming partiel stable et fluide sur le serveur Node-RED.

Enfin, l'interface utilisateur développée offre un outil d'aide à la décision résilient et ergonomique pour le personnel soignant. Grâce aux mécanismes de persistance sous AngularJS, la fonction essentielle de levée de doute reste accessible et exploitable à l'écran, permettant une analyse précise des circonstances d'un incident même après le retour automatique du système à un état de veille stable.

Bien que le dispositif actuel présente des limites intrinsèques liées à la linéarité du suivi ou à la dépendance vis-à-vis des conditions d'éclairage ambiantes, les perspectives d'évolution, telles que l'intégration de capteurs de profondeur 3D ou l'implémentation de filtres de Kalman étendus, tracent une feuille de route claire vers une industrialisation future du système, pour une sécurité accrue au sein des habitats connectés.

Références

[1] C. Drouot, *Conception d'un système de détection des chutes par effondrement pendant la marche pour les patients atteints de Myosite à Inclusions*, Sorbonne Université, 2023. HAL : tel-04079716.

[2] H. T. K. Nguyen et al., *"Low Power Architecture Exploration for Standalone Fall Detection System Based on Computer Vision"*, EMS 2014. HAL : hal-01782703.

[3] F. Hussain et al., *"An Efficient Machine Learning-based Elderly Fall Detection Algorithm"*, 2020. HAL : hal-02528542.

[4] A. R. F. Naushin et al., *"Smart Armband with Fall Detection in Elderly"*, 2025. HAL : hal-05157391.

[5] V. Bazarevsky et al., *"BlazePose: On-device Real-time Body Pose tracking"*, CVPR Workshop, 2020. arXiv : 2006.10204.

[6] I. Craggs, *"MQTT Vs. HTTP for IoT"*, HiveMQ Blog, 2026. Disponible : https://www.hivemq.com/blog/mqtt-vs-http-for-iot/

[7] M. Said Salimo, *"Détection de chute à partir des points du squelette en 2D"*, Medium/Wanabilini, 2023. Disponible : https://medium.com/wanabilini